

1 Суффиксные структуры

1.1 Домашнее задание

1. Для строки длины n найдите усреднённое по всем (не только последовательным) парам суффиксов значение `lcp` с помощью суффиксного массива. $\mathcal{O}(n)$.
2. Дана строка s длины n . Найдите её самую длинную подстроку, разворот которой также входит в s как подстрока, с помощью суффиксного массива. $\mathcal{O}(n)$.
3. Дан набор строк s_i суммарной длины n . Для каждой s_i найдите $\min$ по длине подстроку, которая не встречается в других. $\mathcal{O}(n)$. Решите двумя способами: суффиксными деревом и массивом.
4. Даны k непустых строк суммарной длины n . Найдите за $\mathcal{O}(n)$ среди их общих подстрок p -ю лексикографически. k — не константа.
 - (a) Алфавит константного размера.
 - (b) Алфавит — небольшие целые числа.

Дополнительные задачи

5. Дана строка s длины n . Придумайте структуру данных со следующими операциями:
 - `init(s)` — построить структуру данных. $\mathcal{O}(n \log n)$.
 - `get(p, l, r)` — перечислить вхождения p как подстроки s в позициях от l -ой до r -ой. $\mathcal{O}(|p| + \log n + m)$ (здесь m — размер ответа).
6. Даны словарь непустых строк суммарной длины L , строка s длины n и число p . Пусть X — множество всех строк длины как минимум p , являющихся подстрокой какого-то словарного слова. Найдите в s все позиции, которые не покрыты ни одним вхождением строки из X . $\mathcal{O}(n + L)$.

1.2 Решение

1. Для начала рассмотрим наивное решение данной задачи. Будем перебирать все пары суффиксов и вычислять для них длину общего префикса, в результате чего получим сумму всех lcp и усредним её.

Построим суффиксный массив SA и массив lcp , который хранит длины общих префиксов соседних элементов суффиксного массива.

Используем информацию о том, что суффиксы идут в лексикографическом порядке и, следовательно, для каждой пары суффиксов $SA[i]$ и $SA[j]$ с $i < j$ длина общего префикса $lcp(i, j)$ не может быть больше, чем $lcp(i, i + 1)$.

Тем самым, мы можем посчитать в каком-то смысле вклад каждого $lcp[i]$ в итоговую сумму. Если более формально, то найти такие индексы $L[i]$ и $R[i]$, что $L[i]$ — это индекс ближайшего к i элемента слева, для которого $lcp[L[i]] < lcp[i]$, а $R[i]$ — ближайший справа, $lcp[i] > lcp[R[i]]$. Получаем, что $lcp[i]$ является минимальным элементом на отрезке $lcp[L[i] + 1..R[i] - 1]$ и будет вносить вклад в сумму равный $lcp[i] \cdot (i - L[i]) \cdot (R[i] - i)$.

Таким образом, мы можем посчитать сумму всех $lcp[i]$ по формуле:

$$S = \sum_{i=0}^{n-2} lcp[i] \cdot (i - L[i]) \cdot (R[i] - i)$$

Количество всех пар суффиксов M будет равно $\frac{n(n-1)}{2}$, так как мы перебираем все пары суффиксов, а значит, среднее значение длины общего префикса будет равно:

$$\frac{S}{M} = \frac{2S}{n(n-1)}.$$

Для построения массивов L и R будем использовать монотонный стек. Поддерживаем стек индексов, для которых значения lcp возрастают, если нашли меньший элемент, то выталкиваем из стека элементы, пока не найдём подходящий индекс.

Итоговая сложность данного алгоритма будет равна $O(n)$, так как мы проходим по массиву lcp один раз, а операции с монотонным стеком выполняются за $O(1)$. Построение суффиксного массива и массива lcp также выполняются за $O(n)$.

2. Построим строку $s' = s + \# + s^R$, где s^R — это разворот строки s . Здесь $\#$ — это символ, который не содержится в алфавите строки s .

Построим суффиксный массив для s' и массив lcp для него.

Пройдемся по парам соседних суффиксов $(i, i + 1)$ в суффиксном массиве и проверим, что суффиксы принадлежат разным частям строки s' (один из них начинается с s , а другой с s^R).

Если это так, то $lcp[i]$ будет длиной общей подстроки s и её разворота. Таким образом, мы можем найти максимальное значение $lcp[i]$ среди таких пар и вернуть его.

3. Суффиксное дерево:

- (a) К каждой строке добавим уникальный терминальный символ $\#_i \notin \Sigma$:

$$s_i \rightarrow s_i \#_i.$$

- (b) Построим обобщённое суффиксное дерево (GST) для всех строк $s_1 \#_1, s_2 \#_2, \dots, s_k \#_k$.
- (c) Каждая вершина дерева v соответствует подстроке, которая встречается в множестве строк.
- (d) Для каждой строки s_i найдём вершину, такую что она содержится только в этой строке и её длина минимальна.

Поиск минимальной уникальной подстроки для каждой строки s_i можно выполнить следующим образом:

- Выполняем обход дерева в глубину (DFS), поддерживая текущий путь — подстроку, соответствующую пути от корня до текущей вершины.
- Для каждой вершины v проверяем в каких строках встречается подстрока (за $O(1)$).
- Если только одна строка содержит эту подстроку, то она является кандидатом на минимальную уникальную подстроку.
- Обновляем минимум, если текущая подстрока короче найденной ранее.
- Обход продолжаем по всем вершинам, чтобы гарантировать поиск минимального решения.

Суффиксный массив:

- (a) Объединяем все строки с уникальными терминаторами в строку

$$s' = s_1\#_1s_2\#_2\dots s_k\#_k.$$

- (b) Строим суффиксный массив sa для строки s' и массив lcp .

- (c) Для каждого суффикса $sa[i]$, принадлежащего строке s_j , определяем минимальную длину подстроки, которая не встречается в других строках:

$$l = \max(lcp[i-1] \cdot [owner(sa[i-1]) \neq j], \quad lcp[i] \cdot [owner(sa[i+1]) \neq j]) + 1,$$

где $owner(pos)$ — индекс строки, к которой принадлежит позиция pos .

- (d) Подстрока $s'[sa[i] : sa[i]+l]$ является кандидатом на минимальную уникальную подстроку для s_j .

- (e) Для каждого s_j выбирается подстрока с минимальной длиной.

4. Объединим все k строк в одну строку вида:

$$s' = s_1\#_1s_2\#_2\dots s_k\#_k,$$

где каждый символ $\#_i$ — уникальный терминатор, не входящий в алфавит. Это необходимо для того, чтобы суффиксы из разных строк не пересекались.

Далее:

- (a) Построим суффиксный массив sa и массив lcp для строки s' , используя алгоритм для ограниченного алфавита.
- (b) Для каждого суффикса в sa определим, к какой исходной строке s_i он относится.
- (c) Будем двигать два указателя l и r по суффиксному массиву, поддерживая окно $[l, r]$, которое содержит хотя бы по одному суффиксу от каждой из k строк. Для этого используем скользящее окно и счётчик частот строк внутри окна.
- (d) Для каждого допустимого окна $[l, r]$, покрывающего все k строк, найдём $\min(lcp[l], \dots, lcp[r-1])$ — это длина максимальной общей подстроки, начинающейся с позиции $sa[l]$. Чтобы эффективно получать минимум на отрезке, можно использовать deque.
- (e) Проходим по всем таким окнам в порядке возрастания l и накапливаем число различных общих подстрок. Как только накапливаем p общих подстрок, выводим соответствующую подстроку длины $\min lcp$, начинающуюся с позиции $sa[l]$.